



Data: 18/12/2025

Progetto: OikoNaos	Versione: 1.6
Documento: Object Design	Data: 18/12/2025

Coordinatore del progetto:

Nome	Matricola
Luigi Potestà	0512120076

Partecipanti:

Nome	Matricola
Luigi Potestà	0512120076
Maria Antonietta Ruggiero	0512121126
Giulia Buonafine	0512119695
Mario Sinopoli	0512119113

Scritto da:	Giulia Buonafine
-------------	------------------

Revision History

Data	Versione	Descrizione	Autore
01/12/2025	1.0	Prima versione del documento.	Giulia Buonafine
02/12/2025	1.1	Revisione completa del Purpose, Packages e Class Interfaces Glossary; aggiunta sottosistema Notifiche.	Luigi Potestà
03/12/2025	1.2	Correzione coerenza interna e allineamento con RAD 3.10 e SDD 1.6.	Maria Antonietta Ruggiero
04/12/2025	1.3	Rifinitura formattazione, correzioni linguistiche minori e miglioramento consistenza terminologica; aggiunto diagramma UML dei Package.	Mario Sinopoli
07/12/2025	1.4	Ridefinizione package e Class Interfaces Glossary	Giulia Buonafine
12/12/2025	1.5	Adattamento dei packages alle funzionalità implementate	Luigi Potestà Maria Antonietta Ruggiero
18/12/2025	1.6	Ridefinizione Class Interfaces Glossary adattandola alla versione 1.7 del file System Design	Giulia Buonafine Mario Sinopoli

Indice

1. Purpose	4
2. Audience.....	4
3. Introduction	4
3.1. Object design trade-offs	5
3.2. Interface documentation guidelines	6
3.3. Definitions, Acronyms and Abbreviations	7
3.4. References	7
4. Packages	8
5. Class Interfaces Glossary	10

1. Purpose

Il presente Object Design Document (ODD) descrive la progettazione a oggetti della piattaforma OikoNaos, con particolare riferimento alla versione prototipale iniziale del sistema. Il documento ha l'obiettivo di illustrare come le scelte architetturali definite nel System Design Document (SDD) vengano tradotte in componenti software concreti, organizzati secondo il paradigma Model–View–Controller (MVC).

Il focus dell'ODD è posto sui sottosistemi effettivamente implementati nel prototipo, che costituiscono il nucleo funzionale del sistema. In particolare, la versione corrente di OikoNaos include le funzionalità di autenticazione degli utenti, la gestione delle prenotazioni degli spazi comuni e la gestione dei ticket di manutenzione. Quest'ultima comprende non solo la creazione e la consultazione delle segnalazioni da parte dei coinquilini, ma anche alcune funzionalità avanzate già implementate a supporto del ruolo di Supervisore, quali la presa in carico e l'aggiornamento dello stato dei ticket. Le funzionalità relative alla gestione completa del profilo utente, agli eventi, alle risorse condivise, ai pagamenti e alle notifiche sono previste nell'architettura complessiva del sistema e descritte a livello progettuale, ma non rientrano nel perimetro implementativo della versione prototipale corrente.

Il documento persegue un duplice obiettivo: da un lato fornire ai membri del team una comprensione uniforme della struttura interna del sistema e delle responsabilità delle singole classi; dall'altro garantire la coerenza progettuale tra i requisiti descritti nel Requirements Analysis Document (RAD), l'architettura definita nello SDD e le scelte implementative adottate nel codice effettivamente realizzato.

L'ODD costituisce pertanto un riferimento tecnico fondamentale per lo sviluppo, la verifica e l'evoluzione futura della piattaforma OikoNaos.

2. Audience

Il documento di Object Design Document è rivolto alle figure tecniche coinvolte nello sviluppo, nella verifica e nell'evoluzione della piattaforma OikoNaos, fornendo una descrizione chiara e strutturata delle scelte progettuali adottate a livello di progettazione a oggetti.

- **System Architects:** Utilizzano l'ODD per verificare che la progettazione dei componenti software sia coerente con l'architettura multilivello definita nel System Design Document (SDD) e con i principi di progettazione oggetto del corso, quali elevata coesione, basso accoppiamento e corretta separazione delle responsabilità.
- **Sviluppatori:** L'ODD rappresenta un riferimento operativo per l'implementazione del codice sorgente. Il documento descrive le classi, le loro responsabilità, le dipendenze ammesse e le interazioni tra i componenti, supportando uno sviluppo coerente e allineato alle scelte architetturali.
- **Tester:** Il documento fornisce ai tester le informazioni necessarie per comprendere il comportamento atteso dei componenti software, individuare flussi critici, condizioni di errore ed eccezioni rilevanti, e progettare attività di verifica coerenti con la struttura interna del sistema.
- **Stakeholder dell'Azienda Coordinatrice:** Pur non essendo direttamente coinvolti nelle attività di sviluppo, gli stakeholder possono consultare l'ODD a scopo informativo per comprendere le principali scelte progettuali adottate, con particolare riferimento agli aspetti di sicurezza, gestione dei dati e controllo degli accessi basato sui ruoli (RBAC).

3. Introduction

L'Object Design rappresenta la fase del processo di sviluppo in cui il modello concettuale definito durante l'analisi viene progressivamente trasformato in una struttura software concreta.

In questa fase, i concetti astratti del dominio applicativo vengono mappati in classi, metodi e responsabilità ben definite, introducendo decisioni tecniche e soluzioni progettuali necessarie alla realizzazione del sistema.

Il modello viene arricchito attraverso pattern ricorrenti e componenti appartenenti al dominio della soluzione, come i Data Access Object (DAO) e le classi di supporto alla persistenza dei dati, che consentono di separare la logica applicativa dai dettagli implementativi.

Nel contesto di OikoNaos, l'Object Design è focalizzato sui sottosistemi effettivamente implementati nella versione prototipale iniziale del sistema. In particolare, la progettazione a oggetti riguarda le funzionalità di autenticazione degli utenti, la gestione delle prenotazioni degli spazi comuni e la creazione e consultazione dei ticket di manutenzione. Le funzionalità relative alla gestione avanzata del profilo utente e agli altri sottosistemi previsti dall'architettura complessiva sono considerate parte di evoluzioni future e non rientrano nel perimetro implementativo del prototipo.

Per ciascun sottosistema incluso nel prototipo vengono presentati gli oggetti di tipo Boundary, Control ed Entity, illustrandone il ruolo all'interno del modello e le principali responsabilità. L'organizzazione delle classi è guidata da principi cardine della progettazione orientata agli oggetti, quali la separazione delle responsabilità, l'uso della delega per ridurre l'accoppiamento, la distinzione tra oggetti del dominio applicativo e oggetti introdotti per esigenze implementative, e l'adozione di un'architettura chiusa che protegge i dettagli interni dei componenti.

3.1. Object design trade-offs

Nel processo di progettazione a oggetti della piattaforma OikoNaos sono state effettuate diverse scelte progettuali che comportano compromessi tra obiettivi contrastanti. La presente sezione descrive i principali trade-off considerati, evidenziando le motivazioni alla base delle decisioni adottate nella versione prototipale del sistema.

Sicurezza vs. semplicità di implementazione: Per garantire un livello adeguato di sicurezza nell'accesso alla piattaforma, il sistema adotta meccanismi di autenticazione basati sulla protezione delle credenziali degli utenti, come l'hashing delle password e la gestione controllata delle sessioni. Tali scelte aumentano la complessità dell'implementazione rispetto a soluzioni più semplici, ma risultano necessarie per assicurare la protezione dei dati personali anche nella versione prototipale del sistema.

Riutilizzo di componenti off-the-shelf vs. controllo interno: Nella realizzazione del prototipo sono stati utilizzati componenti e librerie standard per la gestione dell'interfaccia web e di funzionalità di supporto, riducendo il livello di controllo diretto su alcune parti del sistema. Questa scelta consente di concentrare lo sforzo progettuale e implementativo sulle funzionalità centrali del prototipo, quali autenticazione, prenotazioni e ticket di manutenzione, abbreviando i tempi di sviluppo e aumentando la stabilità complessiva.

Manutenibilità del codice vs. tempo di sviluppo: Nonostante i vincoli temporali, la progettazione del sistema segue il paradigma Model–View–Controller e una chiara separazione tra classi Entity, Control e componenti di accesso ai dati (DAO). Tale approccio comporta un incremento del numero di classi e una maggiore articolazione del design, ma migliora sensibilmente la leggibilità del codice, la manutenibilità e la possibilità di estendere il sistema in fasi successive.

Coerenza del modello dati vs. complessità della persistenza: La scelta di modellare entità distinte e ben definite, come Utente, Prenotazione e Ticket di Manutenzione, comporta una struttura del database più articolata, con un numero maggiore di tabelle e relazioni. Tuttavia, questa decisione favorisce la coerenza dei dati, l'integrità referenziale e una rappresentazione chiara dei concetti del dominio,

riducendo il rischio di inconsistenze nelle operazioni di prenotazione e segnalazione.

Prestazioni vs. controlli di integrità nelle prenotazioni: Per evitare situazioni di conflitto, come prenotazioni sovrapposte degli spazi comuni, il sistema esegue controlli di coerenza che coinvolgono più entità del dominio prima di confermare una richiesta. Questo approccio può comportare un lieve aumento dei tempi di risposta, ma garantisce un comportamento affidabile e corretto del sistema, aspetto prioritario per una gestione equa delle risorse condivise.

3.2. Interface documentation guidelines

La documentazione delle interfacce ha l'obiettivo di garantire uniformità, chiarezza e leggibilità all'interno del progetto OikoNaos. A tal fine, la progettazione a oggetti e la relativa documentazione adottano convenzioni sintattiche e stilistiche coerenti, che consentono di identificare immediatamente il ruolo e la natura degli elementi descritti. Nel progetto OikoNaos vengono adottate due principali convenzioni di naming: PascalCase e camelCase, utilizzate in modo sistematico a seconda della tipologia dell'elemento.

La convenzione **PascalCase**, in cui ogni parola inizia con una lettera maiuscola senza l'uso di spazi o caratteri separatori, è utilizzata per tutti gli elementi che rappresentano concetti strutturali del sistema. In particolare, essa è adottata per le classi Entity, per le classi Control, per le classi Boundary e per i componenti di accesso ai dati (DAO). Esempi sono *Utente*, *Prenotazione*, *Ticket*, *AutenticazioneControl*, *PrenotazioneControl*, *AutenticazioneView* e *UtenteDAO*. L'uso di PascalCase consente di riconoscere immediatamente le componenti strutturali del sistema e garantisce un'elevata omogeneità formale all'interno del modello e del codice sorgente.

La notazione **camelCase**, in cui la prima parola è in minuscolo e le successive iniziano con una lettera maiuscola, è utilizzata per i nomi dei metodi, delle variabili di istanza e dei parametri. Esempi di metodi sono *verificaCredenziali()* e *creaPrenotazione()*, mentre esempi di variabili e parametri includono *emailUtente*, *dataPrenotazione*, *idUtente* e *nuovaPassword*. Questa distinzione visiva consente di differenziare chiaramente classi, metodi e attributi, migliorando la leggibilità sia del codice sia della documentazione.

Per garantire coerenza interna nel documento e nell'implementazione, tutti gli identificatori devono rispettare le seguenti linee guida generali. I nomi devono riflettere chiaramente il ruolo della classe o del metodo; in particolare, i controller devono esplicitare l'azione o il processo che coordinano, mentre i metodi devono descrivere il comportamento che implementano. È necessario evitare abbreviazioni non standard, ad eccezione di quelle comunemente riconosciute come DAO, DB e ID. È inoltre preferibile adottare nomi descrittivi piuttosto che nomi eccessivamente brevi, al fine di rendere il codice autoesplicativo. I nomi riportati nella documentazione devono infine essere coerenti con quelli utilizzati nell'implementazione, evitando discrepanze tra progetto e codice.

Al fine di facilitare l'identificazione immediata del livello architetturale di appartenenza, vengono adottati suffissi convenzionali: le classi Boundary terminano con il suffisso *View*, i controller con *Control* e i componenti di persistenza con *DAO*. Questa scelta contribuisce a rendere evidente la separazione delle responsabilità all'interno dell'architettura Model–View–Controller.

Ogni classe descritta nell'Object Design Document deve essere accompagnata da una descrizione sintetica del proprio ruolo, dall'elenco delle principali responsabilità, dai metodi rilevanti con la relativa firma, dalle eventuali eccezioni sollevate e dalle collaborazioni con altre classi del sistema. Le interfacce devono essere definite in modo da riflettere chiaramente lo scopo della classe, senza entrare nei dettagli implementativi, che appartengono alla fase di codifica.

3.3. Definitions, Acronyms and Abbreviations

Termine	Definizione
<i>Co-housing</i>	Modello abitativo in cui più individui condividono spazi e servizi comuni pur mantenendo unità abitative private.
<i>Coinquilino</i>	Utente della comunità residente, abilitato all'utilizzo delle funzionalità principali del sistema.
<i>Azienda Coordinatrice</i>	Ente amministrativo responsabile dell'intera struttura e delle funzioni di supervisione di alto livello.
<i>Supervisore</i>	Utente con permessi avanzati, incaricato di monitorare prenotazioni, ticket e risorse condivise.
<i>Ticket</i>	Segnalazione di guasto o richiesta di intervento tecnico, gestita tramite workflow interno.
<i>DAO (Data Access Object)</i>	Componente che gestisce la comunicazione con il database e implementa operazioni CRUD.
<i>RBAC (Role-Based Access Control)</i>	Modello di autorizzazione basato sui ruoli utente.
<i>Entity Object</i>	Rappresentazione persistente delle informazioni del dominio (es. Utente, Ticket, Prenotazione).
<i>Boundary Object</i>	Oggetto di interfaccia che gestisce l'interazione tra utente e sistema.
<i>Control Object</i>	Oggetto che implementa la logica dei casi d'uso e coordina le operazioni tra View e Model.
<i>TokenResetPassword</i>	Codice temporaneo generato per la procedura di recupero password. Tale funzionalità è prevista a livello progettuale ma non è implementata nella versione prototipale del sistema.
<i>DBMS (Database Management System)</i>	Sistema di gestione del database; in OikoNaos è utilizzato MySQL 8.
<i>HTTPS / TLS</i>	Protocollo di comunicazione cifrato adottato per proteggere lo scambio di dati tra client e server.
<i>JPEG / PNG</i>	Formati di file supportati per allegati a ticket e comunicazioni.

3.4. References

Requirements Analysis_OikoNaos – Versione 3.10: Documento che descrive i requisiti funzionali e non funzionali del sistema, utilizzato come riferimento principale per le scelte progettuali.

System Design_OikoNaos – Versione 1.7: Documento che definisce l'architettura multilivello del sistema, la decomposizione in sottosistemi e i diagrammi UML che costituiscono la base per la progettazione a oggetti descritta nel presente ODD.

4. Packages

La suddivisione del sistema in package rappresenta un passaggio fondamentale nella progettazione di OikoNaos, in quanto consente di ottenere una struttura modulare, coerente e facilmente estendibile. L'obiettivo principale è raggruppare le classi in insiemi omogenei per responsabilità, riducendo l'accoppiamento tra le componenti e facilitando la comprensione dell'architettura complessiva. Seguendo i principi della progettazione orientata agli oggetti, l'organizzazione dei package riflette la distinzione tra oggetti del dominio applicativo, oggetti del dominio della soluzione e ruoli architetturali definiti dal modello Model–View–Controller (MVC). L'architettura è strutturata su tre livelli logici principali: Boundary, Control e Model.

Nella versione prototipale corrente del sistema sono stati inclusi i package relativi alle funzionalità effettivamente implementate nel codice. In particolare, il prototipo implementa i sottosistemi di Autenticazione, Prenotazioni degli spazi comuni, Ticket di Manutenzione per Supervisione e Coinquilino, oltre al package Model per la gestione della persistenza dei dati.

Gli altri sottosistemi previsti dall'architettura complessiva sono considerati parte di evoluzioni future e non sono oggetto di implementazione nella versione attuale. L'architettura a tre livelli prevede che:

- Il livello Boundary comprenda le classi di interfaccia che gestiscono l'interazione con l'utente e inoltrano le richieste ai controller;
- Il livello Control contenga le classi che implementano la logica dei casi d'uso e coordinano il flusso delle operazioni;
- Il livello Model includa le entità persistenti e i componenti di accesso ai dati (DAO) responsabili dell'interazione con il database.

Questa organizzazione realizza un'architettura chiara e stratificata, in cui la comunicazione tra i sottosistemi avviene esclusivamente tramite le View e i Controller, nel rispetto del modello MVC.

Gestione Autenticazione

Il package di autenticazione implementa le operazioni fondamentali di controllo dell'identità dell'utente e costituisce il punto di accesso principale alla piattaforma. Nella versione prototipale sono incluse le funzionalità di registrazione, login e logout, nonché la validazione del codice identificativo fornito all'utente in fase di registrazione. Le funzionalità di recupero della password sono previste a livello progettuale e parzialmente supportate dall'architettura del sistema, ma non rappresentano il focus principale del prototipo.

Categoria	Classe	Descrizione
Boundary	AutenticazioneView	Gestisce l'interazione con l'utente per le operazioni di registrazione, login e logout.
Control	AutenticazioneController	Coordina il processo di autenticazione e verifica le credenziali dell'utente.
Control	RegistrazioneController	Gestisce il flusso di registrazione dei nuovi utenti.
Control	LogoutController	Gestisce la terminazione della sessione utente.
Entity	Utente	Rappresenta l'utente del sistema con i relativi dati anagrafici e di accesso.

Gestione Prenotazioni degli Spazi Comuni

Il package delle prenotazioni gestisce la creazione e la consultazione delle prenotazioni degli spazi comuni e rappresenta una delle funzionalità centrali del sistema. Il sottosistema è progettato per prevenire conflitti e garantire la coerenza delle prenotazioni nel tempo.

Categoria	Classe	Descrizione
Boundary	PrenotazioniView	Espone le funzionalità di visualizzazione delle disponibilità e di creazione delle prenotazioni.
Control	PrenotazioneController	Coordina il processo di creazione e consultazione delle prenotazioni.
Entity	Prenotazione	Rappresenta una prenotazione effettuata da un utente.

Gestione Ticket

Il package dei ticket di manutenzione consente agli utenti di segnalare problemi o guasti e di consultare lo stato delle proprie segnalazioni. La versione corrente del sistema include inoltre funzionalità avanzate di gestione dei ticket, riservate al ruolo di Supervisore, quali la presa in carico delle segnalazioni, l'aggiornamento dello stato e la registrazione delle attività svolte.

Categoria	Classe	Descrizione
Boundary	TicketView	Gestisce l'interazione con l'utente per la creazione e la consultazione dei ticket.
Control	TicketController	Coordina la creazione, consultazione e aggiornamento dei ticket di manutenzione.
Entity	Ticket	Rappresenta una segnalazione di guasto o richiesta di intervento tecnico.

Gestione Supervisore

Il package di supervisione include le funzionalità riservate agli utenti con ruolo di Supervisore, responsabili del monitoraggio e della gestione operativa delle prenotazioni e dei ticket di manutenzione. Questo sottosistema consente l'esecuzione di operazioni avanzate che non sono accessibili ai coinquilini, garantendo il rispetto del modello di controllo degli accessi basato sui ruoli (RBAC).

Categoria	Classe	Descrizione
Control	SupervisoreTicketController	Gestisce le operazioni avanzate sui ticket di manutenzione.
Control	SupervisorePrenotazioniController	Consente il monitoraggio e la gestione delle prenotazioni.

Persistenza dei Dati

Il package Model raccoglie le entità persistenti e i componenti di accesso ai dati utilizzati dai diversi sottosistemi del sistema. Questo livello isola completamente la logica applicativa dai dettagli del DBMS e costituisce il punto di accesso unico al database MySQL. Il package include tutti i DAO del sistema,

inclusi quelli a supporto della gestione avanzata dei ticket, e componenti di supporto per la gestione delle connessioni e delle operazioni di persistenza. Questa separazione favorisce la manutenibilità del codice e consente di evolvere lo schema del database senza impatti diretti sui controller applicativi.

Categoria	Classe	Descrizione
DAO	UserDAO	Gestisce la persistenza e il recupero delle informazioni relative agli utenti del sistema, incluse le operazioni di autenticazione e associazione con prenotazioni e ticket.
DAO	PrenotazioneDAO	Implementa le operazioni di accesso ai dati per la gestione delle prenotazioni degli spazi comuni, incluse le verifiche di disponibilità e il recupero delle prenotazioni associate agli utenti.
DAO	TicketDAO	Gestisce la persistenza dei ticket di manutenzione, supportando le operazioni di creazione, consultazione e aggiornamento dello stato delle segnalazioni.
DAO	AggiornamentoTicketDAO	Gestisce la persistenza degli aggiornamenti associati ai ticket di manutenzione, consentendo la tracciabilità delle attività svolte nel corso del ciclo di vita del ticket.
DAO	AllegatoDAO	Gestisce il salvataggio e il recupero dei file allegati ai ticket di manutenzione, supportando l'associazione tra segnalazioni e contenuti multimediali.

5. Class Interfaces Glossary

Questa sezione descrive le principali classi coinvolte nella versione implementativa della piattaforma OikoNaos, organizzate secondo i tre livelli dell'architettura Model–View–Controller (MVC): View, Controller e Model. Per ciascuna classe vengono illustrate le responsabilità principali, le collaborazioni rilevanti e il ruolo svolto nel flusso applicativo. L'obiettivo è fornire una visione chiara e coerente della struttura interna del sistema, facilitando la comprensione del codice, la manutenzione e la progettazione delle attività di testing

View Layer (Boundary)

Le classi della View rappresentano componenti logici del livello di presentazione. Esse raccolgono l'input dell'utente, visualizzano i risultati delle operazioni e delegano la logica applicativa ai Controller. Tali componenti sono implementati tramite JSP e non necessariamente come classi Java.

Classe	Responsabilità principali	Collabora con
AutenticazioneView	Gestisce le interfacce di login e registrazione e inoltra i dati inseriti al controller di autenticazione.	AutenticazioneController, RegistrazioneController
PrenotazioniView	Visualizza le prenotazioni e consente l'invio di richieste di creazione o consultazione.	PrenotazioneController
TicketView	Consente l'apertura dei ticket, la visualizzazione dello stato e	TicketController, SupervisoreTicketController

	l'interazione con le funzionalità di gestione.	
--	--	--

Controller Layer

I Controller implementano la logica dei casi d'uso e coordinano l'interazione tra View e Model. Nel sistema OikoNaos sono realizzati come Servlet Java.

Classe	Responsabilità principali	Collabora con
AutenticazioneController	Gestisce il login degli utenti e la validazione delle credenziali.	UserDAO
RegistrazioneController	Coordina la registrazione dei nuovi utenti.	UserDAO
LogoutController	Gestisce la chiusura della sessione utente.	—
HomeController	Gestisce l'accesso e il reindirizzamento all'area personale dell'utente autenticato.	—
PrenotazioneController	Coordina la creazione e la consultazione delle prenotazioni degli spazi comuni.	PrenotazioneDAO
TicketController	Gestisce la creazione e la consultazione dei ticket di manutenzione.	TicketDAO, AggiornamentoTicketDAO, AllegatoDAO
SupervisoreTicketController	Gestisce le operazioni avanzate sui ticket riservate al ruolo di supervisore.	TicketDAO, AggiornamentoTicketDAO
SupervisorePrenotazioniController	Consente il monitoraggio delle prenotazioni da parte del supervisore.	PrenotazioneDAO

Model Layer

Il Model include le entità del dominio applicativo e i componenti di accesso ai dati (DAO), che appartengono al dominio della soluzione.

Entity Objects

Classe	Descrizione e ruolo nel sistema
Utente	Rappresenta un utente del sistema con i relativi dati anagrafici e il ruolo associato.
Prenotazione	Descrive una prenotazione effettuata da un utente per uno spazio comune.
Ticket	Rappresenta una segnalazione di manutenzione aperta da un utente.

DAO Objects

DAO	Responsabilità principali
UserDAO	Gestisce la persistenza e il recupero dei dati relativi agli utenti.
PrenotazioneDAO	Gestisce la creazione e il recupero delle prenotazioni.
TicketDAO	Gestisce la persistenza dei ticket di manutenzione.
AggiornamentoTicketDAO	Gestisce la persistenza degli aggiornamenti associati ai ticket.
AllegatoDAO	Gestisce il salvataggio e il recupero degli allegati associati ai ticket.

